FairTune Federated Fine Tuning Proposal

Efficient AI at Scale

Problem 1 Cheap Fine Tuning Across Weak Scattered Devices

Prepared for the hackathon team on 14 October 2026

FairTune will specialize a shared small language model using five clients with different data and compute budgets. This brief contains the proposed abstract, algorithm, architecture, evaluation plan and team demonstration. Training results will be added after implementation and matched experiments.

Project abstract

The abstract contains exactly 900 words excluding its title and this note

Organizations often need to adapt a shared artificial intelligence model using information held across branches, departments, or individual devices. Moving those records into a central training repository may be impractical because of confidentiality requirements, bandwidth constraints, or organizational boundaries. Federated learning addresses data locality by exchanging model updates, but conventional fine tuning can still demand substantial memory and computation. Unequal datasets and processing budgets introduce additional challenges, including inconsistent updates, unstable training, and excessive influence from larger participants. The Efficient AI at Scale challenge therefore requires a practical method that makes limited devices useful while preserving progress and balancing participation.

We propose FairTune, a lightweight federated fine tuning prototype built around a small language model, parameter efficient adaptation, and transparent aggregation controls. The application will classify banking customer queries into service intents, representing branches that collaborate without sending their local training records to the coordinator. Five simulated clients will receive different amounts of data, different intent distributions, and different local computation budgets. Client components will retain their training examples and labels, while the coordinator will receive completed parameter updates and operational statistics. The initial simulation will execute clients sequentially on one laptop, with independent client states and reproducible training schedules.

The shared foundation will be the existing Qwen2.5-0.5B base checkpoint, which contains approximately 494 million parameters. Most model weights will remain frozen. Low Rank Adaptation, or LoRA, will introduce small trainable adjustments in the attention query and value projections, alongside a classification head for 77 banking intents. With the proposed rank of eight, the trainable matrices and head contain 265,600 parameters. Their uncompressed float32 upload is approximately 1.06 megabytes before message overhead. Initial model distribution and recurring communication will be accounted for separately. Actual memory usage, training throughput, and transmitted bytes will be measured rather than inferred from parameter counts.

FairTune will use the established Federated Freeze A LoRA principle to keep the adapter basis consistent across participants. Standard LoRA expresses an adapted weight as a frozen weight plus the product of two smaller matrices, A and B. We will initialize A identically for every client and keep it frozen while training B. Consequently, averaging compatible B updates corresponds directly to averaging their adapter weight updates. This avoids the mismatch that can arise when independently trained factor pairs are averaged separately. The classification head will share the same initial state and participate in aggregation. Clients will use identical adapter shapes.

Project abstract continued

Participation will be balanced through capped square root weighting and update norm clipping. The data allocation will assign 55, 20, 12, 8, and 5 percent of training records to the five clients. Weighting by the square root of dataset size reduces the preference for large datasets, and a cap will prevent any client from receiving more than 35 percent of aggregation weight. The server will normalize weights while respecting that cap, rather than clipping weights and renormalizing incorrectly. Each submitted update will be clipped to a norm bound. These controls limit numerical contribution; predictive fairness will remain an empirical question.

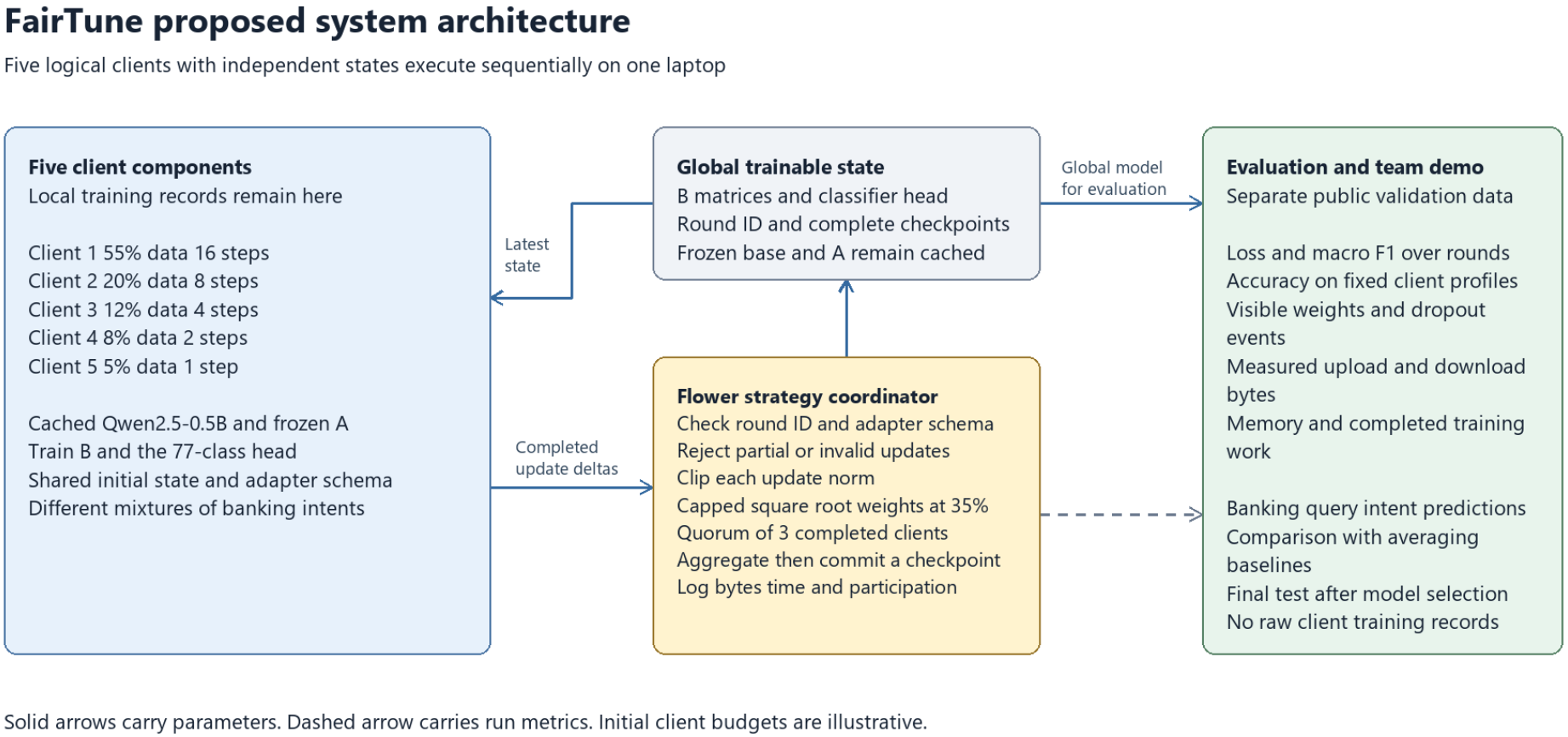
Training will tolerate client interruption through complete update submission, round identifiers, a participation quorum, and checkpoint recovery. Each client will start from the current global adapter and train only within its assigned budget. A client that disconnects before completing its task will contribute no partial update. The coordinator will reject incompatible, stale, or nonfinite updates and aggregate only when at least three valid clients finish. With fewer clients, it will preserve the previous global state and record a skipped round. Returning clients will reload the latest adapter before contributing again. Completed checkpoints will support resuming interrupted experiments without restarting training.

The primary benchmark will be BANKING77, a dataset containing 10,003 training queries and 3,080 test queries across 77 intents. A validation subset will be reserved before constructing client partitions, and the official test split will remain separate from model selection. Client partitions will differ in both size and label mixture to represent statistical heterogeneity. Local optimizer step limits will represent compute heterogeneity through actual differences in training work. A separate CPU benchmark will measure whether a limited client can produce useful updates with the chosen model. Dataset attribution, preprocessing choices, partition seeds, and example overlap checks will accompany the implementation.

Evaluation will compare equal averaging, conventional sample weighted FedAvg, and the proposed capped and clipped aggregation under matched conditions. Every comparison will use the same base checkpoint, adapter configuration, initialization, partitions, local budgets, and participation schedule. We will measure validation loss, macro F1, classification accuracy, performance on fixed client representative evaluation distributions, and the contribution of the smallest participant. Experiments will introduce dropout during training and test client rejoining. Communication measurements will include uploads and downloads, while resource measurements will include peak memory, completed optimizer steps, and round duration. Repeated seeds will quantify variability where the available hackathon time permits.

The prototype targets an RTX 3050 laptop with six gigabytes of graphics memory and approximately sixteen gigabytes of system memory. Compatibility will be confirmed through short benchmarks before longer runs. The team demonstration will combine training curves, visible client weights and status changes, and predictions from the resulting banking query classifier. Its intended outcome is reproducible evidence that limited contributors can improve a shared model despite heterogeneous resources and interruptions. Raw data locality in the simulation will not be presented as a differential privacy guarantee. The submission will distinguish established methods, proposed engineering contributions, measured outcomes, and remaining deployment limitations.

Proposed architecture



The client boundary contains raw training text and labels. Only compatible trainable updates and operational statistics reach the coordinator. The evaluation component holds separate public benchmark data. Five logical clients share the laptop in sequence while retaining independent training states. The diagram has 19 editable Excalidraw elements.

Algorithm and contribution policy

The method is Federated Freeze A LoRA with capped square root weighting and update norm clipping. The frozen adapter basis comes from the published FFA-LoRA method. FairTune adds the participation policy, failure handling, resource budgets and transparent evaluation. The project will credit this prior work and test its limitations.

What trains on each client

The Qwen2.5-0.5B base weights and each common LoRA matrix A remain frozen. Each client trains its B matrices and a bias-free 77-class classification head, starting from the same global state. The initial rank is eight, with adapters on q_proj and v_proj in all 24 attention blocks. Tokenizer settings, LoRA scaling and label order are identical across clients.

Component or payload	Count or size
B matrices in query and value projections	196,608
77-class head with 896 input features	68,992
Total trainable parameters	265,600
FP32 parameter bytes per client upload	1,062,400 bytes

The cached A matrices contain another 344,064 parameters and do not need recurring upload. The upload estimate excludes serialization and message overhead. Initial model distribution, recurring uploads and recurring downloads will be reported separately.

How participation is balanced

For each valid participant, compute the square root of its positive training dataset count. Allocate proportional weights, fix any weight above 0.35 at the cap, and redistribute the remaining weight among uncapped participants. Repeat until the weights sum to one. This is water filling; capping first and blindly renormalizing can violate the cap.

Client	Training data share	Proposed weight
1	55%	35.00%
2	20%	22.36%
3	12%	17.32%
4	8%	14.14%
5	5%	11.18%

These example weights are calculated values, not training results. At least three valid clients are required because two weights capped at 0.35 cannot sum to one. Clipping each update to norm C additionally bounds the norm of any single weighted contribution by 0.35 times C when server_lr is one. Predictive influence and fairness must still be measured.

Round algorithm

```
INITIALIZE
  Cache identical base model W0 and frozen matrices A.
  Initialize global trainable state u = {all B matrices, head}.
  Record the model hash, adapter schema and label map.
  Set C > 0, epsilon > 0 and 0 < server_lr <= 1.
  Pin hyperparameters using validation; default server_lr is 1.

FOR each round t
  Send (u, round_id=t, schema) to scheduled clients.
  Each client starts from u and resets its local optimizer.
  Train for the assigned number of optimizer steps.
  If interrupted, submit nothing for this round.
  If complete, submit delta_i = local_u_i - u,
    positive dataset count n_i and operational statistics.

  S = unique clients with complete, current, finite,
      compatible updates received before the deadline.
  If |S| < 3: log SKIPPED; preserve u; continue.

  For each i in S:
    d_i = delta_i * min(1, C / (norm(delta_i) + epsilon))

  Solve weights with water filling:
    q_i = sqrt(n_i)
    w_i = min(0.35, lambda * q_i), with sum(w_i) = 1

  candidate = u + server_lr * sum(w_i * d_i)
  If candidate is nonfinite: log REJECTED; preserve u.
  Otherwise atomically commit candidate and its round ID.
  Evaluate on separate public validation data.
  Publish metrics, participant weights and run events.

FINALIZE
  Select using validation only; evaluate the sealed test set.
  Export checkpoint, configuration and measured results.
```

Local training and failure rules

An optimizer step means one parameter update after any gradient accumulation. Initial budgets are 16, 8, 4, 2 and 1 steps; microbatch size, accumulation and sequence length will be selected using memory and throughput benchmarks. Local optimizer state is reset at the start of each client task. All clients in a round use the same incoming adapter. Resetting each client to that state prevents sequential execution from becoming chained centralized training.

The server receives no unfinished client update. A reconnecting client fetches the latest committed state and begins a new task. A round deadline, valid-client quorum and checkpoint commit define clear outcomes for dropout. Dataset count metadata assumes registered honest clients; this prototype does not claim resistance to forged identities or malicious model poisoning.

Evaluation and hardware plan

Reserve public validation data from the BANKING77 training split before partitioning client training data. Freeze preprocessing, seeds and evaluation profiles before experiments. Do not use official test performance to choose hyperparameters or checkpoints. Check duplicates and overlap within and across splits; any corrective preprocessing must be documented consistently.

Comparison	Purpose	Conditions held constant
Equal averaging	Simple averaging reference	Same model and adapters, partitions, initialization, optimizer, budgets and participation schedule
Sample-weighted FedAvg	Measure influence of dataset size	Same local training and evaluation setup
Capped and clipped aggregation	Evaluate the proposed server policy	Same local training; change aggregation only
Dropout and rejoining	Measure continuity and skipped rounds	Replay the same failure schedule across methods
Smallest-client ablation	Test whether its distinct data helps	Keep other clients fixed and add an equal-compute control

Report validation loss, macro F1, overall accuracy, mean and minimum accuracy over fixed client representative evaluation profiles, bytes uploaded and downloaded, peak RAM and VRAM, optimizer steps and duration. Evaluation profiles use reserved public benchmark examples and fixed class mixtures; they are not raw client training data. Final test evaluation occurs after selection is sealed. Use three seeds for core comparisons when the available time permits.

Verified development machine

Resource	Verified state	Next check
CPU	Ryzen 5 6600H with 6 cores and 12 threads	Time a real CPU training task
GPU	RTX 3050 Laptop GPU with 6 GB VRAM	Measure peak memory on a short LoRA run
System RAM	15.3 GiB usable; about 1.7 GiB free at inspection	Confirm sufficient free RAM before loading
PyTorch	Current installation is CPU only	Create an isolated CUDA environment

Qwen2.5-0.5B is the primary candidate. SmolLM2-135M is a fallback if measured CPU cost or memory is unacceptable; changing the base model means changing it for every client. The Windows prototype will initially use Flower strategy interfaces with a sequential local harness. Flower documents its Ray simulation support on Windows as experimental.

How the team will see the output

The completed demonstration will use one dashboard with three views. The learning view compares validation loss and macro F1 over rounds. The participation view shows all five clients, their assigned budgets, completed steps, weights, bytes and dropout events. The prediction view accepts a banking query and displays the predicted intent from a selected trained checkpoint. Logged experiment replays will be labeled as replays; a live run will display its actual run ID and progress.

Demo time	Action	Evidence to show
0 to 1 minute	Explain the architecture and task	Five local partitions and shared cached model
1 to 2 minutes	Show the trained-model comparison	Measured curves, macro F1 and client profile scores
2 to 3 minutes	Inspect the client with 55% of data	Its recorded weight is capped at 35%
3 to 4 minutes	Trigger one client dropout and rejoin	Completed updates continue; stale work is rejected
4 to 5 minutes	Try held-out banking queries	Predictions from the selected checkpoint
5 to 6 minutes	Show cost and reproducibility	Measured bytes, memory, configurations and seeds

Completion order

Complete the abstract and architecture first. Then prepare the isolated CUDA environment, dataset and model metadata; benchmark one GPU task and one CPU task; build the baseline simulation; implement the server policy and dropout handling; run the controlled experiments; and assemble the dashboard, checkpoints and final submission. Core claims will be updated only after the corresponding measurements exist.

Sources and method attribution

Qwen2.5-0.5B model card and checkpoint metadata

BANKING77 dataset card and CC BY 4.0 attribution

Improving LoRA in Privacy-preserving Federated Learning

Hugging Face PEFT LoRA developer guide

Flower simulation and Windows support documentation